

TECHNOLOGY OF COMPILERS LAB REPORT



IMPLEMENTATION OF SYNTAX ANALYZER

Name: IGBO KENNETH CHIAGOZIE

ID: 5112100502

Class: COMPUTER SCIENCE 3RD YEAR 2023/2024 SESSION

Teacher: NIU, DE JIAO

Date: May 26, 2024

PURPOSE AND CONTENT

1. Implement and understand the aspect and implementation of syntax analyzers.
2. Understand the concept of parsing trees and parsing tables.

CONTENT

1. Source codes and their respective outputs.
2. Explanations and illustrations of ideas used.

SYNTAX ANALYSIS

Syntax analyzers are also known as parsers are also an important component of compilers and interpreters. It takes the output tokens from the lexical analyzers and checks the correctness of the source codes' grammatical structure. This process is known as PARSING.

SYNTAX ANALYSIS PROCESS

In this lab experiment I used the Bottom Up SLR(1) parser. This is a parser that processes the input from left to right and produces a rightmost derivation in reverse. It also uses a parsing technique called Shift-Reduce parsing which involves the shifting of tokens onto a stack and reduces them to non-Terminals using stated production rules.

1. Lexical analysis:

This is the lexical analysis of the source code which gives the output of Tokens.

2. Syntax Analysis:

This is the syntax analysis of the source code where using the SLR(1) parser, the source code is checked for correctness of structure.

TOKEN TYPES

1. Integers
2. Identifiers
3. Symbols
4. Keywords

```
keyword_list = ["or", "and", "not", "true", "false"]  
symbol_list = ["(", ")", ";"]
```

IMPLEMENTATION

1. Token Class:

This is the classes that represents tokens.

```
class Token:
    def __init__(self, in_type, value, symbol_table=None):
        self.type = in_type
        self.value = value
        self.table_pos = 0
        self.symbol_Table = symbol_table

    def __str__(self):
        classified = self.classify_value()
        if classified is not None:
            return f'<{repr(self.value)}>'
        if self.type == INTEGER:
            self.table_pos = self.symbol_Table.find_in_constant_table(self.value)
            if self.table_pos == -1:
                self.table_pos = self.symbol_Table.add_to_constant_table(self.value)
        else:
            self.table_pos = self.symbol_Table.find_in_variable_table(self.value)
            if self.table_pos == -1:
                self.table_pos = self.symbol_Table.add_to_variable_table(self.value)
        return f'<{self.type}, {repr(self.table_pos)}>'

    def __repr__(self):
        return str(self.value)

1 usage
def classify_value(self):
    if self.value in keyword_list:
        return "keyword"
    elif self.value in symbol_list:
        return "symbol"
    return None
```

2. Symbol Tables Class:

This is the class that denotes the variable and constant table.

```
class SymbolTables:
    def __init__(self):
        self.variable_table = []
        self.constant_table = []
        self.variable_table_length = 0
        self.constant_table_length = 0

    2 usages (2 dynamic)
    def add_to_variable_table(self, value):
        self.variable_table.append([self.variable_table_length + 1, value])
        self.variable_table_length += 1
        return self.variable_table_length

    2 usages (2 dynamic)
    def add_to_constant_table(self, value):
        self.constant_table.append([self.constant_table_length + 1, value])
        self.constant_table_length += 1
        return self.constant_table_length

    2 usages (2 dynamic)
    def find_in_constant_table(self, value):
        for i in self.constant_table:
            if i[1] == value:
                return i[0]
        return -1

    2 usages (2 dynamic)
    def find_in_variable_table(self, value):
        for i in self.variable_table:
            if i[1] == value:
                return i[0]
        return -1
```

3. Lexer/Lexeme Class:

This is a class that denotes and performs the identification, recognition and classification of lexemes.

```
class Lexer:
    def __init__(self, text):
        self.text = text
        self.pos = 0
        self.current_char = self.text[self.pos]
        self.flag = False

    1 usage
    def error(self, type=1):
        if type == 1:
            raise Exception(f'Invalid character {self.current_char}')

    4 usages
    def advance(self):
        self.pos += 1
        if self.pos > len(self.text) - 1:
            self.current_char = None
        else:
            self.current_char = self.text[self.pos]

    def retract(self):
        self.pos -= 1
        if self.pos > len(self.text) - 1:
            self.current_char = None
        else:
            self.current_char = self.text[self.pos]

    1 usage
    def skip_whitespace(self):
        cond = self.current_char in symbol_list
        while self.current_char is not None and (self.current_char.isspace() or self.current_char == "\n") and not cond:
            self.advance()
            cond = self.current_char in symbol_list

    def integer(self):
        result = ''
        while self.current_char is not None and self.current_char.isdigit():
            result += self.current_char
            self.advance()
        if self.current_char is not None and self.current_char.isalpha():
            self.current_char = result + self.current_char
            # self.error()
            return "error"
        return int(result)

    1 usage
    def string(self):
        result = ''
        while self.current_char is not None and self.current_char.isalnum():
            result += self.current_char
            self.advance()
        if self.current_char in symbol_list:
            self.flag = True
        return result

    2 usages
    def get_next_token(self, symbol_Table):
        while self.current_char is not None:
            if self.current_char.isspace() or self.current_char == "\n":
                self.skip_whitespace()
                continue

            if self.current_char.isdigit():
                temp_ret = self.integer()
                if temp_ret != "error":
                    return Token(INTEGER, temp_ret, symbol_Table)

            if self.current_char.isalpha():
                return Token(IDENTIFIER, self.string(), symbol_Table)

            if self.current_char in symbol_list:
                temp_ret = self.current_char
                self.advance()
                return Token(SYMBOL, temp_ret)

            self.error()
            break

        return Token(EOF, value=None)
```

4. states Class:

This is a class that denotes the states in the SLR(1) parsing table.

```
class states:
    def __init__(self, state_value):
        self.state = state_value

    def __str__(self):
        return str(self.state)

    def __int__(self):
        return int(self.state)
```

5. Productions Class:

This is a class that represents the productions of the grammar.

```
class productions:
    def __init__(self, pr_head, pr_body=None, nitems=None):
        self.head = pr_head
        self.body = pr_body
        if self.body is not None:
            self.nitems = len(self.body)

    def __str__(self):
        new_str = ""
        for i in self.body:
            new_str += repr(i)
        return f"{self.head}-> {new_str}"

    def __int__(self):
        return int(self.nitems)

    def __repr__(self):
        return self.head
```

6. Parsing Stack Class:

This is a class that denotes the parsing stack used in the SLR(1) parsing.

```
class parsing_stack:
    def __init__(self):
        self.parsing_stack = [Token(SYMBOL, value: "#"), states(0)]
        self.parsing_stack_length = 2

    4 usages
    def push(self, token):
        self.parsing_stack.append(token)
        self.parsing_stack_length += 1

    1 usage
    def pop(self, number_of_times=0):
        if self.parsing_stack_length == 0:
            return None
        if number_of_times == 0:
            self.parsing_stack_length -= 1
            return self.parsing_stack.pop()
        else:
            n_times = number_of_times * 2
            last_object = None
            for i in range(n_times):
                self.parsing_stack_length -= 1
                last_object = self.parsing_stack.pop()
            return last_object

    2 usages
    def peek(self):
        if self.parsing_stack_length == 0:
            return None
        return self.parsing_stack[-1]

    def get_all(self):
        display_string = ""
        if self.parsing_stack_length == 0:
            return display_string
        for i in self.parsing_stack:
            if isinstance(i, Token):
                display_string += i.value
            elif isinstance(i, states):
                display_string += str(int(i))
            elif isinstance(i, productions):
                display_string += repr(i)
        return display_string

    1 usage
    def original_stack(self):
        self.parsing_stack = [Token(SYMBOL, value: "#"), states(0)]
        self.parsing_stack_length = 2

    1 usage
    def clear(self):
        self.parsing_stack = []
        self.parsing_stack_length = 0
```

7. detection_thread function:

This is the function which performs the detection of the input in the GUI editor text box. It runs on a new thread. It ensures that the compilation and analysis of the input is only done when there is a change registered. It also allows for pausing and continuation of detection thread.

```
def detection_thread():
    global output_textbox, input_textbox, client_queue
    textbox_string = ""
    client_queue = mp.Queue()
    global prev_token_list
    prev_token_list = []
    while True:
        check_queue()
        if textbox_string != input_textbox.get(index1: 1.0, index2: "end-1c"):
            symbol_Table = SymbolTables()
            token_list = []
            textbox_string = input_textbox.get(index1: 1.0, index2: "end-1c")
            output_textbox.config(state="normal")
            output_textbox.delete(index1: 1.0, index2: "end")
            output_textbox.insert(index: 1.0, chars: f"{analyze(textbox_string, symbol_Table, token_list, prev_token_list)}")
            output_textbox.config(state="disabled")
            variable_textbox.config(state="normal")
            variable_textbox.delete(index1: 1.0, index2: "end")
            variable_textbox.insert(index: 1.0, chars: f"{get_variable_table_text(symbol_Table)}")
            variable_textbox.config(state="disabled")
            constant_textbox.config(state="normal")
            constant_textbox.delete(index1: 1.0, index2: "end")
            constant_textbox.insert(index: 1.0, chars: f"{get_constant_table_text(symbol_Table)}")
            constant_textbox.config(state="disabled")
```

8. analyze function:

This is the function used to analyze the input token and is called by the detection thread function. It gets the tokens, and uses them to update the symbol and constant table accordingly. Afterwards it returns the analyzed text which are tokens. If there is no error in the lexical analysis, it starts a new process for the syntax analysis.

```
def analyze(text, symbol_Table, token_list, prev_token_list):
    global new_proc
    analyzed_text = ""
    lexer = Lexer(text + "\n")
    error_flag = False
    try:
        token = lexer.get_next_token(symbol_Table)
        if token.type != EOF:
            token_list.append(token)
    except Exception as e:
        analyzed_text += f"Error at {str(e)[18:]} \n"
        token = Token(EOF, value="")
        error_flag = True

    while token.type != EOF:
        analyzed_text += str(token) + "\n"
        try:
            token = lexer.get_next_token(symbol_Table)
            if token.type != EOF:
                token_list.append(token)
            if repr(token) == ";":
                token_list.append(hash_token)
        except Exception as e:
            error_flag = True
            analyzed_text += f"Error at {str(e)[18:]} \n"
            break

    if not error_flag and token_list != prev_token_list:
        prev_token_list = token_list
        try:
            if new_proc.is_alive():
                new_proc.kill()
        except Exception as e:
            pass
        new_proc = mp.Process(target=syntax_analysis, args=(token_list, output_table, client_queue))
        new_proc.start()

    return analyzed_text
```

9. syntax_analysis function:

This is a function that performs syntax analysis on the tokens from the lexical analyzer. In this code, it runs on a new process.

```
def syntax_analysis(token_list, output_table, client_queue):
    parse_stack = parsing_stack()
    c_point = 0
    counter = 1
    try:
        clear_table(output_table)
    except Exception as e:
        pass
    global parsing_steps_table
    action = "None"
    while c_point < len(token_list) and token_list[c_point] is not None:
        current_state = parse_stack.peek()
        input_token = token_list[c_point]
        action = action_parsing_table.at[int(current_state), repr(input_token)]
        new_row_to_insert = []
        if repr(action) == "None":
            new_row_to_insert = [f"{parse_stack.get_all()}", f"{repr(input_token)}",
                                f"Error At {repr(input_token)}, state= {int(current_state)}"]
            output_table.loc[len(output_table.index)] = new_row_to_insert
            break
        elif repr(action) == "SHIFT":
            prev_token = input_token
            parse_stack.push(input_token)
            c_point += 1
            parse_stack.push(states(int(action)))
            new_row_to_insert = [f"{parse_stack.get_all()}", f"{repr(input_token)}",
                                f"Shifted to state {int(action)} ",
                                f"due to input {repr(prev_token)}"]
            output_table.loc[len(output_table.index)] = new_row_to_insert
        elif repr(action) == "REDUCE":
            prev_token = input_token
            c_prod = production_lists[int(action)]
            last_elem = parse_stack.pop(int(c_prod))
            temp_state = parse_stack.peek()
            action = goto_parsing_table.at[int(temp_state), repr(c_prod)]
            parse_stack.push(c_prod)
            parse_stack.push(states(int(action)))
            new_row_to_insert = [f"{parse_stack.get_all()}", f"{repr(input_token)}",
                                f"Reduced {str(c_prod)} and new state is ",
                                f"{int(action)} due to input {repr(prev_token)}"]
            output_table.loc[len(output_table.index)] = new_row_to_insert
        elif repr(action) == "ACCEPT":
            parse_stack.clear()
            new_row_to_insert = [f"{parse_stack.get_all()}",
                                f"{repr(input_token)}", f"Accepted Current String"]
            output_table.loc[len(output_table.index)] = new_row_to_insert
            parse_stack.original_stack()
            c_point += 1
            counter += 1
    if repr(action) != "None" and repr(action) != "ACCEPT":
        new_row_to_insert = [f"{parse_stack.get_all()}", f"{repr(input_token)}",
                            f"Error At {repr(input_token)}, state= {int(current_state)}"]
        output_table.loc[len(output_table.index)] = new_row_to_insert
    client_queue.put(output_table)
```

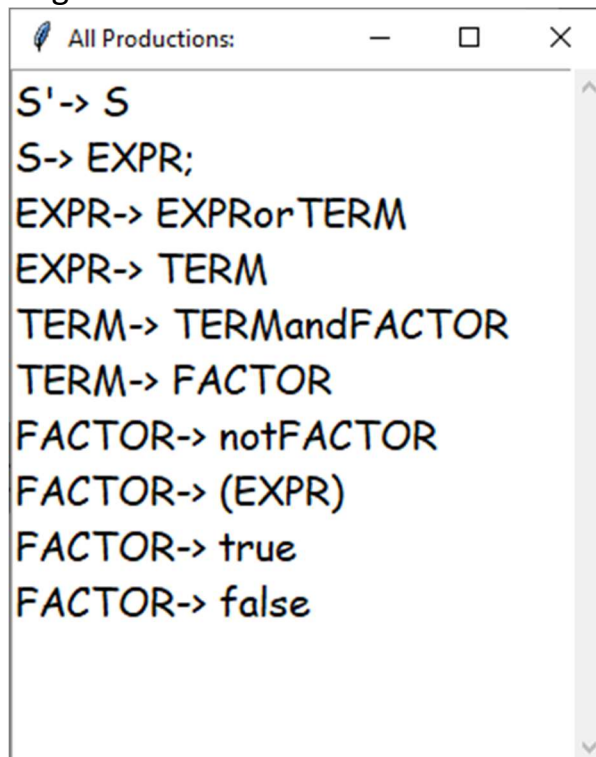
10. Main function:

This is the main function and denotes the start of the code execution. It creates the file path, initializes the threading event and instruction queue. Afterwards it creates and initializes the GUI and starts the detection thread.

```
if "__main__" == __name__:  
    createPath()  
    global event, instruction_queue, paused  
    event = threading.Event()  
    instruction_queue = queue.Queue()  
    paused = False  
    root = tk.Tk()  
    initGUI(root)  
    detection()  
    root.mainloop()
```

PRODUCTIONS AND PARSING TABLE

The productions for the grammar are as follows:



```
All Productions:  
S' -> S  
S -> EXPR;  
EXPR -> EXPR or TERM  
EXPR -> TERM  
TERM -> TERM and FACTOR  
TERM -> FACTOR  
FACTOR -> not FACTOR  
FACTOR -> (EXPR)  
FACTOR -> true  
FACTOR -> false
```

It is the grammar for the Boolean expressions.

The parsing table was derived by me and is as follows:

	or	and	not	(	)	true	false	;	#
1			SHIFT, 5	SHIFT, 6		SHIFT, 7	SHIFT, 8		
2									ACCEPT, None
3	SHIFT, 10							SHIFT, 9	
4	REDUCE, 3	SHIFT, 11			REDUCE, 3			REDUCE, 3	
5	REDUCE, 5	REDUCE, 5			REDUCE, 5			REDUCE, 5	
6			SHIFT, 5	SHIFT, 6		SHIFT, 7	SHIFT, 8		
7			SHIFT, 5	SHIFT, 6		SHIFT, 7	SHIFT, 8		
8	REDUCE, 8	REDUCE, 8			REDUCE, 8			REDUCE, 8	
9	REDUCE, 9	REDUCE, 9			REDUCE, 9			REDUCE, 9	
10									REDUCE, 1
11			SHIFT, 5	SHIFT, 6		SHIFT, 7	SHIFT, 8		
12			SHIFT, 5	SHIFT, 6		SHIFT, 7	SHIFT, 8		
13	REDUCE, 6	REDUCE, 6			REDUCE, 6			REDUCE, 6	
14	SHIFT, 10				SHIFT, 16				
15	REDUCE, 2	SHIFT, 11			REDUCE, 2			REDUCE, 2	
16	REDUCE, 4	REDUCE, 4			REDUCE, 4			REDUCE, 4	
17	REDUCE, 7	REDUCE, 7			REDUCE, 7			REDUCE, 7	

Action Table

	S	EXPR	TERM	FACTOR
1	MOVE, 1	MOVE, 2	MOVE, 3	MOVE, 4
2				
3				
4				
5				
6				MOVE, 12
7		MOVE, 13	MOVE, 3	MOVE, 4
8				
9				
10				
11			MOVE, 14	MOVE, 4
12				MOVE, 15
13				
14				
15				
16				
17				

GOTO TABLE

The tables and productions listed above can be easily displayed by using the buttons on the GUI interface to display them. Also, they can be exported to a file accordingly and I will include one.

SLR(1) PARSING PROCESS

To perform SLR(1) parsing, we first derive the action and goto table as has been done above. Afterwards we start reading in tokens.

We need to keep track of the current pointed token, the current state and the contents of the parsing stack.

The rows on the tables indicate states while the columns on the action table are input tokens while on goto table indicate non-Terminals.

With the current token and current state, we find their intersection on the table.

If the intersection is empty; then there is an error and the current grammar is incorrect and an error is called until corrected.

If the intersection is SHIFT, number ; this indicates that we need to shift the current input and current state into the stack i.e push the current input token and state into the stack. Afterwards, we move to the next token in the list of input tokens and the new state is the number indicated by the intersection.

If the intersection is REDUCE, number ; this indicates that we need to reduce the contents in the parsing stack using the production rule with the number indicated at the intersection. To accomplish this, the contents of the stack are popped accordingly and the head of the production rule is pushed into the stack. The new state is derived by taking the intersection of the production head and most recent state in the stack in the GOTO table.

If the intersection is accept; This indicates that the current string is accepted and this is displayed on the output table. This clears the stack and starts it for the next string if there is.

TESTING AND OUTPUT

The testing and output pictures and results are included below:

Source string:

true or false;

(true and false or(true or false and false or true) and true or false) or false;

Lexical Analyzer

Editor:

```
true or false;
(true and false or(true or false
and false or true) and true or f
alse) or false;
```

Output Lexemes:

```
<'true'>
<'or'>
<'false'>
<','>
<'('>
<'true'>
<'and'>
<'false'>
<'or'>
<'('>
<'true'>
<'or'>
<'false'>
<'and'>
<'false'>
<'or'>
<'true'>
<'('>
<'true'>
<'and'>
<'true'>
<'or'>
<'false'>
<'('>
<'or'>
<'false'>
<','>
```

Symbol Tables:

Variable / Identifier Table:

Pos	Variable Names
-----	----------------

Constant Table:

Pos	Constant Values
-----	-----------------

Parsing Steps:

	Current Stack	Current Input To	Current Action
5	#0EXPR2or10	or	Shifted to state 10 due to input or
6	#0EXPR2or10false8	false	Shifted to state 8 due to input false
7	#0EXPR2or10FACTOR4	:	Reduced FACTOR-> false and new state is 4 due to input
8	#0EXPR2or10TERM14	:	Reduced TERM-> FACTOR and new state is 14 due to input
9	#0EXPR2	:	Reduced EXPR-> EXPRorTERM and new state is 2 due to input
10	#0EXPR2,9	:	Shifted to state 9 due to input ;
11	#0S1	#	Reduced S-> EXPR, and new state is 1 due to input #
12	#	#	Accepted Current String
13	#0(6	(	Shifted to state 6 due to input (
14	#0(6true7	true	Shifted to state 7 due to input true
15	#0(6FACTOR4	and	Reduced FACTOR-> true and new state is 4 due to input a
16	#0(6TERM3	and	Reduced TERM-> FACTOR and new state is 3 due to input
17	#0(6TERM3and11	and	Shifted to state 11 due to input and
18	#0(6TERM3and11false8	false	Shifted to state 8 due to input false
19	#0(6TERM3and11FACTOR15	or	Reduced FACTOR-> false and new state is 15 due to input
20	#0(6TERM3	or	Reduced TERM-> TERMandFACTOR and new state is 3 due to input
21	#0(6EXPR13	or	Reduced EXPR-> TERM and new state is 13 due to input
22	#0(6EXPR13or10	or	Shifted to state 10 due to input or
23	#0(6EXPR13or10(6	(	Shifted to state 6 due to input (
24	#0(6EXPR13or10(6true7	true	Shifted to state 7 due to input true
25	#0(6EXPR13or10(6FACTOR4	or	Reduced FACTOR-> true and new state is 4 due to input o
26	#0(6EXPR13or10(6TERM3	or	Reduced TERM-> FACTOR and new state is 3 due to input
27	#0(6EXPR13or10(6EXPR13	or	Reduced EXPR-> TERM and new state is 13 due to input
28	#0(6EXPR13or10(6EXPR13or10	or	Shifted to state 10 due to input or
29	#0(6EXPR13or10(6EXPR13or10false	false	Shifted to state 8 due to input false
30	#0(6EXPR13or10(6EXPR13or10and	and	Reduced FACTOR-> false and new state is 4 due to input
31	#0(6EXPR13or10(6EXPR13or10and	and	Reduced TERM-> FACTOR and new state is 14 due to input
32	#0(6EXPR13or10(6EXPR13or10and	and	Shifted to state 11 due to input and
33	#0(6EXPR13or10(6EXPR13or10false	false	Shifted to state 8 due to input false
34	#0(6EXPR13or10(6EXPR13or10and	or	Reduced FACTOR-> false and new state is 15 due to input
35	#0(6EXPR13or10(6EXPR13or10and	or	Reduced TERM-> TERMandFACTOR and new state is 14 due to input
36	#0(6EXPR13or10(6EXPR13or10and	or	Reduced EXPR-> EXPRorTERM and new state is 13 due to input
37	#0(6EXPR13or10(6EXPR13or10and	or	Shifted to state 10 due to input or
38	#0(6EXPR13or10(6EXPR13or10and	true	Shifted to state 7 due to input true
39	#0(6EXPR13or10(6EXPR13or10and	)	Reduced FACTOR-> true and new state is 4 due to input)
40	#0(6EXPR13or10(6EXPR13or10and	)	Reduced TERM-> FACTOR and new state is 14 due to input
41	#0(6EXPR13or10(6EXPR13or10and	)	Reduced EXPR-> EXPRorTERM and new state is 13 due to input
42	#0(6EXPR13or10(6EXPR13or10and	)	Shifted to state 16 due to input)
43	#0(6EXPR13or10(6FACTOR4	and	Reduced FACTOR-> (EXPR) and new state is 4 due to input

Pause Detection and Analysis

Save Files Display Parsing Trees Display Productions

From here we can see the outputs. The files containing each output and source file will be available with the report.

For a simpler example, we can use the following string:

Source string:

true and false;

Lexical Analyzer

Editor:

```
true and false;
```

Output Lexemes:

```
<'true'>  
<'and'>  
<'false'>  
<';'>
```

Symbol Tables:

Variable / Identifier Table:

Pos	Variable Names
-----	----------------

Constant Table:

Pos	Constant Values
-----	-----------------

Parsing Steps:

	Current Stack	Current In	Current Action
1	#0true7	true	Shifted to state 7 due to input true
2	#0FACTOR4	and	Reduced FACTOR-> true and new state is 4 due to input and
3	#0TERM3	and	Reduced TERM-> FACTOR and new state is 3 due to input and
4	#0TERM3and11	and	Shifted to state 11 due to input and
5	#0TERM3and11false8	false	Shifted to state 8 due to input false
6	#0TERM3and11FACTOR15	;	Reduced FACTOR-> false and new state is 15 due to input ;
7	#0TERM3	;	Reduced TERM-> TERMandFACTOR and new state is 3 due to input ;
8	#0EXPR2	;	Reduced EXPR-> TERM and new state is 2 due to input ;
9	#0EXPR2,9	;	Shifted to state 9 due to input ;
10	#0S1	#	Reduced S-> EXPR, and new state is 1 due to input #
11		#	Accepted Current String

Pause Detection and Analysis

Save Files Display Parsing Trees Display Productions

CONCLUSION

The syntax analyzer is another important step in the compilation of source code. It ensures that all grammatical rules are followed and ensures that the code is safe to run or compile to the target language accordingly.

There are other parsers such as LR(0), LR(1), SLR(0), etc. These have some differences from the SLR(1) used in this lab but they are all very important given the circumstances.